

Solvimon API — Fonctionnement du pipeline

Détail technique des 3 scripts du bot « Solvimon API » (duplication d'abonnements en masse), pour revue par les équipes techniques.

Périmètre — solvimon_api/ **Mis à jour le** — 8 septembre 2026

SOMMAIRE

- | | |
|-------------------------------|---------------------------------------|
| 1. Vue d'ensemble du pipeline | 4. 2 — Vérifier la Subscription |
| 2. Configuration partagée | 5. 3 — Créer les Subscriptions |
| 3. 1 — Créer les Customers | 6. Points de vigilance opérationnelle |

· Vue d'ensemble du pipeline

Les 3 scripts s'enchaînent manuellement, dans cet ordre, pour dupliquer un abonnement Solvimon existant vers un lot de nouveaux clients.

ÉTAPE 1

Créer les Customers

Scrape l'admin PayPlug, crée les fiches client (`POST /customers`) côté Solvimon.

ÉTAPE 2

Vérifier la Subscription

Lecture seule — confirme que l'abonnement « source » à dupliquer existe bien.

ÉTAPE 3 · SENSIBLE

Créer les Subscriptions

Duplique l'abonnement pour chaque client et l'active — facturation réelle déclenchée.

· Configuration partagée aux 3 scripts

- **Authentification** — deux clés distinctes dans `.env` : `SOLVIMON_API_KEY_PROD` et `SOLVIMON_API_KEY_SANDBOX`, envoyées en header `X-API-KEY`. La clé est sélectionnée selon l'environnement choisi au lancement ; si celle-ci est vide, le script s'arrête avant tout appel.
- **Choix d'environnement** — `prod` (`payplug.solvimon.com`) ou `sandbox` (`test.api.solvimon.com`), sélectionné manuellement à chaque exécution (terminal ou dashboard).
- **DRY_RUN** — variable d'env `DRY_RUN=1` ou paramètre dashboard. Exécute toute la navigation/les requêtes de lecture, mais court-circuite les appels d'écriture (`POST / PATCH`) sur `create_customers.py` et `solvimon_bulk_subscriptions.py` — le payload est loggué au lieu d'être envoyé.
- **TEST_MODE** — limite le run à 1 seule ligne et bascule Chromium en mode visible ; n'affecte pas les appels API (indépendant du DRY_RUN).
- **Retry / backoff / timeout** — chaque appel Solvimon (GET/POST/PATCH) a un timeout de 20s et est automatiquement retenté jusqu'à 3 fois en cas de `429`, `5xx` ou erreur réseau (timeout, connexion), avec backoff exponentiel (1s/2s/4s) ou respect du header `Retry-After` s'il est présent.
- **Idempotency-Key** — dérivée d'identifiants stables (company_ref pour la création de customer, source+customer pour la copie d'abonnement), pas d'un identifiant aléatoire — un retry sur la même ligne ne duplique pas la ressource côté Solvimon.

1 Créer les Customers — `create_customers.py`

Entrée : `input/admin_ids.csv` (colonne `id`).

Séquence, pour chaque ID

1. Ouvre Chromium avec la session Admin PayPlug déjà sauvegardée (`admin_payplug/session.json`).
2. Va sur `admin.payplug.com/admin/companies/{id}` et scrape le DOM : raison sociale, company_ref, adresse, code postal, ville, SIRET, TVA, nom commercial, email.
3. Vérifie que `company_ref`, `SIRET`, `TVA` et `email` sont bien renseignés — si l'un manque, la ligne est marquée en erreur et **aucun appel n'est envoyé à Solvimon**.
4. Construit le payload et exécute `POST /customers` : type `ORGANIZATION`, reference = company_ref, email, `organization.legal_name`, `organization.tax_id` (TVA), `organization.registration_number` (SIRET), adresse complète.

- 5. Erreurs (scraping, validation ou HTTP) journalisées par ligne ; le run continue sur les IDs suivants.
- 6. Log CSV → `results/results_create_customers_*.csv` (colonnes : `company_id`, `company_ref`, `raison_sociale`, `nom_commercial`, `siret`, `email`, `solvimon_id`, `status`, `message`).

2 Vérifier la Subscription — `inspect_subscription.py`

LECTURE SEULE Aucun effet de bord. `GET /pricing-plan-subscriptions/{sub_id}` sur l'environnement choisi ; affiche statut, `customer_id`, plan en clair dans le terminal. Sert de vérification manuelle avant l'étape 3.

3 Créer les Subscriptions —

`solvimon_bulk_subscriptions.py`

ÉTAPE SENSIBLE

Entrée : deux sources possibles, au choix au lancement :

SOURCE	DÉTAIL
<code>customers.csv</code> (manuel)	Colonnes <code>id</code> (ID du compte input) et <code>customer_id</code> , remplies à la main.
Résultats récents — Créer les Customers	Réutilise directement un fichier <code>results_create_customers_*.csv</code> de l'étape 1 : colonne <code>company_id</code> → <code>id</code> , colonne <code>solvimon_id</code> → <code>customer_id</code> , ne conserve que les lignes <code>status == OK</code> .

Séquence

- 1. Vérifie l'abonnement source (`GET`) et affiche clairement son statut, le **nom** du customer auquel il est *actuellement* affecté et le **nom** de l'offre (résolus via des appels complémentaires `GET /customers/{id}` et `GET /pricing-plans/{id}`, best effort — repli sur les IDs bruts si indisponibles) — arrête tout le run si introuvable.
- 2. Charge les lignes à traiter depuis la source choisie.

- **3.** Si l'exécution est réelle (pas de DRY_RUN) : affiche le nombre de clients concernés et demande une confirmation explicite avant de continuer — en terminal via un prompt o/N, ou depuis le dashboard via un bouton Oui/Confirmer qui apparaît directement dans la fenêtre de suivi du run.
- **4.** Pour chaque ligne : `POST /pricing-plan-subscriptions/{source_id}/copy` avec une référence `{id}-{customer_id}-{timestamp}` .
- **5.** Puis `PATCH /pricing-plan-subscriptions/{new_id}` avec `{customer_id, status: "ACTIVE"}` — active immédiatement le nouvel abonnement.
- **6.** Erreur sur un client → journalisée, le run continue sur les suivants.
- **7.** Log CSV → `results/results_solvimon_*.csv` (colonnes : id, customer_id, status, subscription_id, message).

Confirmation interactive depuis le dashboard. Le pipe stdin du script reste ouvert côté serveur pendant l'attente de confirmation ; le frontend détecte un marqueur émis par le script et affiche un bouton Oui/Non qui écrit la réponse directement dans le process en cours. Si la fenêtre de suivi est fermée avant confirmation, le run reste en pause côté serveur jusqu'à réponse ou redémarrage du dashboard.

Points de vigilance opérationnelle

- **Pas de rollback automatique.** Un run interrompu en cours de route (erreur, arrêt manuel) laisse le système dans un état mixte — certains clients activés, d'autres non — reconstituable seulement à la main via le CSV de résultats.
- **Clé API prod à renseigner.** `SOLVIMON_API_KEY_PROD` doit être remplie dans `.env` avant tout run réel en environnement `prod` ; le script bloque sinon avec un message explicite.
- **Scraping couplé au DOM Admin PayPlug.** `create_customers.py` dépend de sélecteurs sur les pages admin — un changement de front côté Admin PayPlug peut faire échouer le scraping (capturé en erreur par ligne, ne bloque pas le run).

Document généré à partir de l'analyse du code de `solvimon_api/` (bot-updater).